
title: "Windows 10 on External USB SSD: The Correct Manual Way"
summary: "A great blog from decryptingtechnology.blogspot.com
revamped" date: 2026-07-31T13:20:32+03:00 draft: false tags:
["Windows", "USB", "Diskpart", "DISM", "Hardware"] categories:
["Systems"] author: batcain math: false

This blog is a shout-out to <https://decryptingtechnology.blogspot.com/2015/09/install-windows-10-on-usb-external-hard.html>. It is the greatest blog about subject. Search engines became trash and it became very hard to find this blog when I need it. It also required some size revisions after 11 years. If you follow it blindly, typos and small partition sizes takes TOO MUCH TIME to troubleshoot and fix. So here we are.

Why Windows does not allow it?

For short explanation, it is a whiner. And here is the long answer:

Windows Kernel Architecture soft blocks it for boot device reliability and memory management reasons. In a default setup, disk should be loaded with `SERVICE_BOOT_START (0X0)` priority. A disk connected through USB port is either loaded with `SERVICE_SYSTEM_START(0X1)` or `SERVICE_DEMAND_START (0X3)`. Therefore when kernel wants to mount the boot volume, the USB stack is not initialized yet.

Also if there is a tiny bit of a movement and disconnect in USB port, obviously, everywhere would be flashing with BSOD and kernel could not recover from those page faults. It is not a very stabilized port to run bloatware OS, as you could imagine.

Memory manager would expect atomic reads, writes and would not be able to get it from USB bulk transfers.

It might sound bad when put this way, but it is not as bad that the drama queen Windows tells us. It is a very reasonable way to use Windows if you are only using it for stupid certification proctoring spyware and if you do not want to reset your host linux for a stupid 60 minute exam.

Why manual partition it?

Well, because setup.exe does everything very abstract to the user to keep them out of the loop which includes IOCTL queries made to your external driver. IOCTL sends `IOCTL_STORAGE_QUERY_PROPERTY` query to receive `_STORAGE_DEVICE_DESCRIPTOR` struct of your connected disk.

```
typedef struct _STORAGE_DEVICE_DESCRIPTOR {
    ULONG          Version;
    ULONG          Size;
    UCHAR          DeviceType;
    UCHAR          DeviceTypeModifier;
    BOOLEAN        RemovableMedia;
    BOOLEAN        CommandQueueing;
    ULONG          VendorIdOffset;
    ULONG          ProductIdOffset;
    ULONG          ProductRevisionOffset;
    ULONG          SerialNumberOffset;
    STORAGE_BUS_TYPE BusType;
    ULONG          RawPropertiesLength;
```

```
    UCHAR                RawDeviceProperties[1];  
} STORAGE_DEVICE_DESCRIPTOR, *PSTORAGE_DEVICE_DESCRIPTOR;
```

Your SSD connected through USB port would most likely return `RemovableMedia = TRUE` and setup would not consider your disk as an eligible disk and eliminates it from setup disks. Obviously UI is not the way to go through.

What does it change?

After a successful setup, you get the

`HKLM\SYSTEM\CurrentControl\SetControl\PortableOperatingSystem` registry set to 1. Which means you won't be able to use the following in your portable Windows:

1. Major updates (Big relief, great)
2. Hibernate (Because who really needs it)
3. Some store applications (Literally no one needed ever)

Overall great, no one needed those. Also I could not care less about those pain in the ass updates on the no longer maintained OS. So, yay!

One last note about WIM, SWM and ESD Files

For small setup ISO's we had monolithic `.wim` (Windows Imaging) files. Since everything needs to be bloatware, FAT32 partition in setup disk cannot handle `.wim` files larger than 4GB. Therefore those larger `.wim` files require a logical split and a new file format. For this reason we get to know about `.swm` (Split WIM/Multi-segment WIM) files.

These multi swm files has a master-slave relationship, first `install.swm` file is the master and holds WIM header, how many parts (slaves) there is, metadata resource table and compressed file part (until it's total size is 4GB ofc). Slaves has the rest of compressed file parts. All of them combined is an actual imaging file.

Commands

I had 256GB SSD, so my sizes are adjusted according to my disk. Change necessary values of your disk specs.

- Create a windows 10 USB bootable disk and boot from it.
- Once you're in the Setup program, select your language, time and currency format and input method, and click Next. Click the Install Now button. Enter/Skip your Windows key if prompted, and read and accept the software licence. In the next screen, press "SHIFT+F10" to open **command prompt**. Also connect your external hard disk.
- Now type diskpart and create the partition using the following script below

DISKPART

```
diskpart
```

```
list disk

select disk x (where x your disk number 0,1,2,3,... and so on)

clean (This will format the whole disk)

convert gpt

create partition primary size=350

format quick fs=ntfs label="Windows RE Tools"

assign letter="T"

set id="de94bba4-06d1-4d40-a16a-bfd50179d6ac"

gpt attributes=0x8000000000000001

format quick fs=fat32 label="System"

assign letter="S"

create partition msr size=128

create partition primary size=256000
** NOTE: I've given 250GB Disk space to the drive where windows will be
installed, change this value according to your disk size **

format quick fs=ntfs label="Windows"

assign letter="W"

create partition primary size=4096

format quick fs=ntfs label="Recovery Image"

assign letter="R"

set id="de94bba4-06d1-4d40-a16a-bfd50179d6ac"

gpt attributes=0x8000000000000001

**NOTE: I've a 500GB External hard disk and partitioned it accordingly.
After this I get an unallocated disk space roughly say 245 GB which I can
create a partition here using "create partition primary" or later after
Windows installation using Disk Management. **

list volume

exit
```

```
md R:\RecoveryImage

(C = check usb LTR on Windows diskpart list volumes)

copy C:\sources\install*.swm R:\RecoveryImage

dism /Apply-Image /ImageFile:R:\RecoveryImage\install.swm
/SWMFile:R:\RecoveryImage\install*.swm /Index:1 /ApplyDir:W:\

md T:\Recovery\WindowsRE

copy W:\Windows\System32\Recovery\winre.wim T:\Recovery\WindowsRE\winre.wim

bcdboot W:\Windows /s S: /f UEFI

W:\Windows\System32\reagentc /setosimage /path R:\RecoveryImage /target
W:\Windows /index 1

W:\Windows\System32\reagentc /setreimage /path T:\Recovery\WindowsRE
/target W:\Windows
```